



PDF Download
3769994.3770023.pdf
01 April 2026
Total Citations: 0
Total Downloads: 360

 Latest updates: <https://dl.acm.org/doi/10.1145/3769994.3770023>

RESEARCH-ARTICLE

“A River of Tears Is a Valid Answer?”: Social Science Graduate Students’ Experiences Learning Programming for Research

ALEXIS TARTER, University of Minnesota Twin Cities, Minneapolis, MN, United States

MOYAN ZHOU, University of Minnesota Twin Cities, Minneapolis, MN, United States

SVETLANA YAROSH, University of Minnesota Twin Cities, Minneapolis, MN, United States

DAVID DELIEMA, University of Minnesota Twin Cities, Minneapolis, MN, United States

Open Access Support provided by:

University of Minnesota Twin Cities

Published: 10 November 2025

Citation in BibTeX format

Koli Calling '25: 25th Koli Calling
International Conference on Computing
Education Research
November 11 - 16, 2025
Koli, Finland

“A River of Tears Is a Valid Answer?”: Social Science Graduate Students’ Experiences Learning Programming for Research

Alexis Tarter
University of Minnesota
Minneapolis, MN, USA
tarte007@umn.edu

Svetlana Yarosh
University of Minnesota
Minneapolis, MN, USA
lana@umn.edu

Moyan Zhou
University of Minnesota
Minneapolis, MN, USA
zhou0972@umn.edu

David DeLiema
University of Minnesota
Minneapolis, MN, USA
ddeliema@umn.edu

Abstract

Computing is a key skill needed in today’s increasingly digital world. However, for novice learners outside of computer science programs, the experience of learning to program can elicit “a river of tears,” as a graduate student provocatively reported in our recent interview study. Seventeen graduate students from the social sciences participated in semi-structured interviews focused on how they acquire programming skills, the barriers they face, and the resources they wish they could access. Through data-driven thematic analysis, we found common values underlying social science graduate students’ desired learning supports including availability, transferability, and connection. Future learning strategies and supports to ameliorate social science graduate students’ experiences could focus on resolving tensions between values, minimizing the wide disparity in resources among social science graduate students, and increasing the transferability support structures and the connections between social science graduate students, peers, and instructors. This research project contributes empirical understandings of social science graduate students’ experiences learning to program for their independent research.

CCS Concepts

• **Social and professional topics** → **Computing literacy**.

Keywords

Computing Education, Research Computing, Scientific Computing, Higher Education

ACM Reference Format:

Alexis Tarter, Moyan Zhou, Svetlana Yarosh, and David DeLiema. 2025. “A River of Tears Is a Valid Answer?”: Social Science Graduate Students’ Experiences Learning Programming for Research. In *25th Koli Calling International Conference on Computing Education Research (Koli Calling ’25)*, November 11–16, 2025, Koli, Finland. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3769994.3770023>



This work is licensed under a Creative Commons Attribution 4.0 International License. *Koli Calling ’25, Koli, Finland*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1599-0/25/11

<https://doi.org/10.1145/3769994.3770023>

1 Introduction

As datasets expand and research questions cross disciplinary boundaries, scholars in the social sciences require increasingly advanced computational tools and expertise. Traditionally, point-and-click tools such as SPSS¹ have sufficed to accomplish the data management, analysis, and visualization tasks required for quantitative research. Yet as early as 2010, Nolan and Temple Lang [39] “call for computing concepts to be integrated into the statistics curricula at all levels” (p. 97). With the increasing availability of data and the growth of data science over time, social science researchers need more robust software tools [27]. For instance, R, an open-source programming language, is free for users and enables researchers to conduct complex analyses and visualizations through highly customizable functions and packages. While highly adaptive and powerful, these robust programming tools also require significant training to be effective and efficient, potentially placing a new burden on burgeoning scholars, namely graduate students.

Within the social sciences, the need for teaching computational skills for statistical tasks such as data management, manipulation, and visualization continues as scholars face critically limited investigations into teaching strategies (see endnote 2, [30]). Graduate students often build research skills from their departmental curriculum and advisor mentorship [41]. However, a major shift to programming statistical tools may weaken these common pillars of graduate student support. Programming instruction for social science undergraduate and graduate students is often lacking, even when programming software is required [30], and faculty in scientific fields, such as Environmental Sciences, report a lack of confidence in mentoring graduate students on computing [52]. As traditional pathways remain limited, social science graduate student training could benefit from further study into innovative and effective programming support.

While research in computing education has investigated numerous interventions for CS major and non-major undergraduates students, the goals and experiences of social science graduate students differ significantly. Well-researched pedagogical frameworks [26], tools [24, 42], and strategies [12, 15, 19, 37, 43] have been developed for introductory programming courses within computer science (CS) programs. More recently, the unique goals [53] and motivations of non-major undergraduate students for personal rather

¹<https://www.ibm.com/products/spss-statistics>

than career interests [29] have been studied. Major and non-major undergraduate students also report differences in their experiences within CS introductory courses [48] including specific fears and varying confidence levels [28]. When learning to code in introductory courses, non-major undergraduate students across a variety of disciplines report valuing clear alignment of course content with personal interests, active learning strategies, and strong connection with their peers [1]. In response to these needs and experiences, tailored introductory CS course designs have been investigated and developed for non-major undergraduate students [14, 22]. While graduate student programmers in the social sciences may share some qualities with non-major undergraduate students, such as novice proficiency and extracurricular motivations, there are also many differences. Graduate student programmers in the social sciences are often learning programming in tandem with another subject and have highly specific goals related to their research.

Considering these differences, scholars have recently begun key research on scientific computing pedagogies in statistics curriculum [34] and computational social science curriculum [32]. Student support offices and university libraries are also investigating short-term workshops and training sessions [25, 30, 52] to teach research computing skills. Graduate students outside of computer science have been found to successfully learn to code through self-taught strategies, working closely with a singular expert, and utilizing community knowledge through collaborations with peers [51] while reporting common feelings of isolation and varying levels of success in building a community of learners on their own [40]. This nascent, and as of yet limited, body of work calls for further investigation into course and career development for teaching and learning programming in the social sciences.

In contribution to a growing field of study, we conducted semi-structured interviews to investigate specific strategies graduate students in the social sciences use to gain programming skills for independent research. We focus on three specific research questions:

- **RQ1:** How do graduate students in the social sciences report gaining coding skills?
- **RQ2:** What challenges do graduate students in the social sciences describe encountering while acquiring coding skills?
- **RQ3:** What resources do graduate students in the social sciences wish they had to acquire coding skills?

We used data-driven thematic analysis without a priori hypothesized themes since there is little previous literature on graduate student learning of computing skills for research within the social sciences. In the following sections, we discuss our positionality as researchers, review key literature on social science graduate student computing education, describe our methods, report our results, and highlight key insights, limitations, and implications for future research.

1.1 Positionality Statement

As a PhD candidate studying educational technologies, the first author has personal experience learning coding to complete their own social science coursework and research. They also work as an Academic Technologist for a large liberal arts college helping instructors design online learning environments. For this project, the

first author's positionality as a graduate student and staff member provides multiple benefits including additional context and access. Specifically, their experience in the workplace informs a greater understanding of the ecosystem of computing education for statistics within social sciences. Their status as a graduate student reinforces their empathy for the experiences of the participants and opens key avenues for recruitment. However, their roles also have the potential to bias their interpretations towards their own lived experiences. To mitigate potential biases, the first author stated their positionality as a staff member at the beginning of each interview. Additionally, a second reviewer - a PhD candidate in computer science- joined the project for data analysis to expand the research team's perspectives.

2 Related Works

In this section, we briefly review the development of computing education, goals, and limitations of support for non-major programmers as well as the pedagogies for and learning experiences of programmers beyond CS courses.

2.1 Development of Computing Education for Non-Major Programmers

At the post-secondary level, computer science became an independent discipline as early as the late 1950s with significant curricular development in the following decades [2]. Liberal arts and humanities colleges began developing computer science programs as early as the 1980s [17]. Since then, researchers continue to highlight the key role liberal education plays in computing from the origin of computer science degree program developments [6] to current innovations [50].

Over time, computing education research has identified effective pedagogical frameworks, tools, and strategies to teach programming skills within computer science courses. 'Hedy' borrows the pedagogical framework used to teach punctuation to create a gradual programming language [26]. Multiple tools support student learning through code execution and debugging [24] as well as highlighting components of code [42]. Instructors have implemented numerous strategies within the classroom: block interfaces [43], analogies [12], explain-in-plain-English exercises [15], and feedback structures [19, 37]. For CS major undergraduate students, many interventions exist for teaching programming.

Researchers have also focused on the unique characteristics and needs of non-major undergraduate students taking CS courses. Major and non-major undergraduate students report differing experiences in the classroom [48]. Non-major undergraduate students report nine specific fears when entering introductory CS courses related to coding, comprehension, being left behind, perceiving STEM as difficult, managing workload, grading, preparation, disappointment in the course, and no fears. While non-major undergraduate students report comparable levels of confidence at the start of the course, those students who report fears of coding, lack of preparation, and being left behind have the highest gains of confidence [28]. Various course designs also show positive impacts for non-major undergraduate students including CS0 course designs that incorporate clearly articulated motivations, processes for feedback, relevant programming languages, and learning strategies supported

by educational sciences [22] as well as CS 0.5 courses that include relevance for unique interests, alignment with CS major curriculum, tailored pacing, encouraged positivity, and scalable practices [14]. Multiple supports for learning, such as a block-based visual tool [54] and a browser-based language subset tool [4], as well as classroom strategies, such as non-major peer mentoring [46] and increased assignment difficulty [13], show improvements for non-major undergraduate students in introductory CS courses. While well-researched within formal computer science programs for majors and non-major undergraduate students, pedagogy focusing on computing skills for students in the social sciences is still limited.

2.2 Goals of Non-Major Programmers in Higher Education

Beyond traditional career goals, such as software engineers, non-major undergraduate students taking CS courses have different and specific goals compared to CS major undergraduate students. Non-major undergraduate students cite their own interests rather than career goals as their motivation for taking computing courses [29]. One emerging line of work, led by Guzdial and Evrard [21], investigated the computing needs and current teaching offerings in the College of Literature, Science, and the Arts (LSA) at University of Michigan. They collaborated with a small, representative task force of faculty, recruited additional faculty to interview across the college, and identified the following goals: computing for discovery, computing for expression, and computing for justice/critical computing. Computing for discovery focuses on the ways students and faculty use computing to create and evaluate new knowledge, particularly through methods of automation, modeling, and simulation. Computing for expression allows students and faculty to engage others through digital media. Computing for justice/critical computing highlights the need for critical thought and action. In contrast to the often technical objectives of primary and low-secondary introductory computer science lessons such as abstraction, decomposition, algorithm design, and pattern recognition [31], liberal arts faculty identify introductory computing needs at a collegiate level centered on non-technical skills and applications. Our study addresses strategies to teach differing end goals as indicated in prior work.

In addition to broader liberal goals for discovery, expression, and critical computing needs, Nolan and Temple Lang [39] "call for computing concepts to be integrated into the statistics curricula at all levels" (p. 97). Disciplines across the sciences are more reliant than ever on computing skills as datasets grow larger and statistical procedures become more sophisticated. NASA, for example, predicted a six-fold increase from 40 petabytes in 2021 to 245 petabytes in 2025 for their earth science program alone [11]. Researchers working with these growing datasets for climate science require training and competency in coding to complete key statistical processes including accessing and managing institutional databases, plotting data to troubleshoot and visualize models, and validate analyses [45]. Researchers in the life and physical sciences also require programming for data analysis and visualization [7]. Social scientists are no different. Even over a decade ago, the social sciences grappled with ever-growing datasets and digital research environments

[27] which emphasize the need for strong computing skills to complete rigorous, transparent, and reproducible statistical analyses. Therefore, rather than becoming coding experts themselves, social science researchers focus on unique endpoints, namely data management and analysis [53]. Situated learning models recognize that skills grow in the context of specific activities with unique infrastructures; this situated view of learning transfer [23] makes a strong case for social sciences-specific forms of computing teaching and learning. While introductory pathways for gaining expert knowledge have long been recognized in computer science [36], limited attention has been given to identifying effective pedagogy for graduate-level computing skills within the social sciences.

2.3 Limitations of Support for Non-Major Programmers

Significant limitations currently exist within traditional graduate education structures of support, namely curriculum and mentorship, for computing education outside of computer science programs. While statistical courses often require graduate students to obtain and use computational tools such as R, STATA, or SAS, they infrequently include the learning of those tools in the course outcomes [30]. The lack of focus on programming in course objectives may indicate limited curricular attention, especially if instructors design their coursework from these objectives. When asked why graduate students are not learning research computing skills, faculty in fields such as Environmental Science have stated that they often do not have instructors with enough expertise to teach the skills and some faculty even stated feeling "personally out of touch with [students] because I haven't taken the time to learn R, because of my training and age" [52] (p. S87). With infrequent inclusion in course objectives and limited faculty knowledge in computing, initial evidence suggests that graduate students are offered minimal opportunities to learn computational skills for research through statistical curriculum or the mentor-mentee relationship with advisors. Outside of the classroom, there is an "unspoken expectation" that graduate students gain computing skills on their own [25] (p. 184). Beyond the classroom and advisor mentorship, student support offices and university libraries offer workshops to teach research computing skills [25, 30, 52]. While various workshops have been researched, designed, and evaluated, the pedagogy included in workshops have not been well-researched. Our work addresses this gap by gaining understanding of social science graduate student experiences navigating support systems including workshops.

2.4 Pedagogies for and Educational Experiences of Programmers beyond CS Courses

Beyond computer science classrooms, various educational strategies have been developed for undergraduate students in the social sciences and humanities. Guzdial and their research team [20] created Task-Specific Programming (TSP) languages, or teaspoon languages, to introduce a small portion of programming hyper-focused on a single-task into secondary history and social studies courses. Since the target population is not graduate level students, we cannot claim that the specific teaspoon language lesson would seamlessly transfer to post-secondary contexts. However, with lessons only taking minutes to learn, 'teaspoon' size activities may help undergraduate

students to learn the logic and syntax of coding with direct application to their specific disciplines before further post-secondary work. Graduate-level workshops have been implemented for both discipline-agnostic [3] and discipline-specific contexts including those focused on environmental science [52], education [49], and sociology [30]. In addition to workshops, researchers have studied various pedagogical frameworks for teaching programming outside of computer science such as a "big picture and workflow" [32] (p. 606) process that first states the intended patterns of action and then discusses the precise steps needed. In application, teachers following this process begin by stating the intended actions through programming fundamentals, then move into identifying the specific steps of wrangling messy data and communicating data analyses through visualizations. Other researchers call for integrating computing and statistics education principles when teaching programming for research (e.g. reviewing both inference and predictive models) [34]. By balancing these two approaches, instructors can provide graduate students with more comprehensive understandings of programming and data science.

While still a growing line of inquiry, burgeoning work in statistical education and computing education has begun to articulate the learning experiences for programmers outside of computer science majors. Li and their team [35] describe the negative emotions and engagement behaviors of non-major undergraduate students and found their negative experiences can lead to disengagement from course assignments including procrastinating, delaying effort until office hours, or receiving partial credit. However, regulation (e.g. planning assignments) and disciplinary (e.g. debugging, asking for help) strategies can lead to re-engagement or prevent disengagement from homework. Scholars also find that undergraduate students learning to program in STEM and education fields experience significant struggles and make more errors in R than their computer science peers [44]. Focusing on researchers' experiences when programming, scholars studied within the life and physical sciences report using digital tools such as ChatGPT, Stack Overflow, and GitHub Copilot to aid with coding [7]. Graduate level environmental science students report additional strategies to learning to program: utilizing independent learning resources through research projects, collaborating with peers, and connecting with a local expert [51]. O'Brien and Mick's [40] interview study highlights the learning experience of researchers across disciplines and positions. Including, but not limited to, graduate students in the social sciences, they similarly found that researchers often attempted to learn programming independently but encountered significant barriers, including common feelings of isolation. While some participants reported efforts to build their own learning communities, they achieved varying levels of success. Our study furthers understanding of current strategies social science graduate students use to learn programming for research, as well as illuminates design values and priorities for future resource designs.

3 Methods

In our study, we conducted semi-structured interviews with social science graduate students about the learning strategies they use, the barriers they face, and the resources they wish they could access.

In the following sections, we describe our participants, explain our data collection procedure, and review our data analysis process.

3.1 Participants

We interviewed seventeen graduate students from multiple disciplines at a large, R1, land-grant midwestern university. We recruited participants through emails, flyers, and instructor's word of mouth announcements in introductory statistical, programming, and research courses that utilize programming languages within social science departments. Emails, flyers, and instructors' word of mouth announcements were also sent to R programming workshops offered through an academic support office for the liberal arts. Additionally, a flyer announcement was posted in a graduate student council newsletter which is sent to all fee-paying graduate students across the institution. Snowball recruitment methods were also used to encourage further participation from graduate students eligible for the study. During recruitment, each participant completed a questionnaire to screen for eligibility based on the following criteria: (1) participants were enrolled in at least one credit and had an active student status; (2) participants self-identified as non-STEM students²; (3) participants had not previously completed a formal degree in computer science. Participants were also required to be working on a research project using coding or working on a class project with the intention of using coding for future research projects.

All participants included in the study represent the social sciences with extensive representation in fields of psychology. The specificity of participants within the social sciences is both expected and justified as quantitative research using large datasets and robust data analysis has become highly prevalent within the social sciences [27] and relies on strong programming knowledge. Participants were recruited across all graduate levels to capture students most likely working on their own research projects. While some undergraduate students may be working on research projects, they are often closely supervised whereas graduate students are training to become independent researchers themselves. The narrow breadth and scope of our participants has limitations which are further articulated in our discussion.

Table 1 shows the demographic information of the seventeen participants interviewed, fourteen of whom reported taking a statistics or programming course that required writing code to complete the course assessments. Over half (12) of the participants study in psychology-related programs which require two introductory statistics courses that mandate the use of R. Three participants did not report any formal coding coursework within their program when describing resources they utilized when gaining coding skills. Regardless of formal classroom training, participants reported needing coding skills to complete data collection, organization, management, analysis, and visualization. Skill levels of coding also greatly varied. Even though no participants had obtained a formal degree in computer science, some had never seen or attempted to code previous to

²We used non-STEM as an eligibility criteria to allow for a wider range of participants within the social sciences as many social science graduate students self-identify with a variety of terms (e.g. behavioral sciences, liberal arts and sciences, developmental sciences, etc). We defined non-STEM for recruitment as fields outside of biological, mathematical, engineering and technological sciences. However, three students within mathematical and engineering fields self-reported as non-STEM and were excluded from the study.

Table 1: Participant Characteristics

ID	Student Status	Year	Project	Field	Languages Used
P2	PhD Student	1st	No	School Psychology	R
P3	PhD Student	4th	Yes	Educational Psychology	R, Python, JavaScript
P4	PhD Candidate	> 6th	Yes	Education	R, Python
P5	PhD Student	1st	No	Developmental Psychology	R
P6	PhD Student	2nd	Yes	Anthropology	R, Python
P9	PhD Student	1st	No	Family Sciences	R
P10	PhD Student	2nd	Yes	Social Psychology	R
P11	PhD Candidate	1st	Yes	Developmental Psychology	R, Python
P12	PhD Candidate	5th	Yes	Psychology	R
P13	PhD Candidate	6th	Yes	Social Psychology	R
P14	PhD Student	2nd	No	Education	R
P15	PhD Candidate	4th	Yes	Educational Psychology	R
P16	PhD Student	5th	Yes	Educational Psychology	R, JavaScript
P17	PhD Student	4th	Yes	Educational Psychology	R, Python
P18	PhD Candidate	4th	Yes	Developmental Psychology	R
P19	PhD Student	3rd	Yes	Development Psychology	R
P20	PhD Student	1st	Yes	Developmental Science	R

their graduate program while other participants had taken multiple courses for programming or held a professional position that uses programming. Regardless of the amount of coding skill developed before entering their graduate program, every participant was expected to accomplish a higher level of coding during their programs than they had previously demonstrated prior to enrollment.

3.2 Data Collection

After participants signed up for an interview time slot through a google calendar, we invited them to complete the preliminary questionnaire to screen for eligibility in the study and record previous education and training in computing. In the questionnaire, we asked participants to review the consent form and indicate if they are willing to participate, identify basic demographic information, list their research experience and educational history in coding and programming, identify languages and software used to write code, and state if any previous degrees in computer science had been completed. Once participants completed the preliminary questionnaire and were deemed eligible for the study, we invited them to an interview over Zoom. During the interview, we provided participants a copy of the consent form and we reviewed the form together. Upon a verbal confirmation of agreement, we asked the participant if they would be comfortable with the interview being recorded. All participants except P4 agreed to have the Zoom meeting recorded during the interview. P4 declined recording the Zoom meeting and we took extensive hand-written notes. We then asked a series of semi-structured questions focused on the participant's research project history, current skill pathways and barriers, current coding skill resources, and desired coding resources. The interviews ranged from 19 to 85 minutes ($\bar{x}$ = 45.44 mins, s = 18.64 mins). We concluded data collection once we had representation from multiple disciplines across the social sciences, reached a higher number of participants than the local average of 15

[10] for similar methods, and began to find saturation [16] through repeated responses without new contributions from participants.

3.3 Data Analysis

We analyzed the interviews through a reflexive, data-driven thematic analysis with inductive coding [8]. From an experiential approach, we analyzed participants' interview responses as a record of their lived experiences learning to program [9]. The audio-recordings of the interviews on Zoom were automatically converted into a text transcript through the recording software for all participants except P4. To analyze P4's interview we typed up our extensive interview notes and coded the notes in the same way as the transcripts for other participants. No quotes were cited for P4 due to the limitations of the interview notes. However, the participant's perspective is reflected in the data analysis through coding and clustering. We conducted reflexive, data-driven thematic analysis, as a team, using the reviewed text transcripts and typed notes. First, we familiarized ourselves with the data by reviewing the auto-generated transcripts, editing them for accuracy, and reviewing notes from P4's interview. Next, we re-read the transcripts and open-coded one transcript together to establish common practices for analysis. We then divided the remaining transcripts and notes to record open codes (smallest units of meaning) and memos (general notes of relevant context). After all of the transcripts and notes were coded, we developed themes by comparing each code against all other codes to identify its relation to developing clusters. Once all of the codes were inductively placed into candidate themes, we then reviewed each theme for further refinement by combining, expanding, and separating as needed. Finally, we defined common values through "identifying the 'essence'" [8] of the themes and organized these values into three main categories, each with two to three sub-themes (p. 92). For each sub-theme, we captured direct quotations and confirmed their accuracy by re-reviewing the original video recordings. We articulated arising tensions between values

through Value Sensitive Design by recognizing potential trade-offs [38]. As values developed, we identified "features that even a small number of stakeholders view as particularly problematic" (p. 284) and highlighted desired attributes.

4 Results

Building from previous computing education research, our study identifies how participants report learning to program, how they describe barriers to learning, and the resources they wished they were able to access (see Figure 1). As participants spoke about these strategies, barriers, and wishes, we noticed underlying values. Through our data-driven thematic analysis, we identified three main value themes, each with two to three sub-themes: Availability including Timeliness, Awareness, and Accessibility; Transferability including Specificity, Workability, and Trustworthiness; and Connection including Camaraderie with peers and Guidance from experts.

4.1 Availability

Availability refers to the ability of a resource to be used or accessed. Students need to be available to use resources and resources need to be available for use. Participants referred to three particularly salient components of availability: Timeliness, Awareness, and Accessibility.

4.1.1 Timeliness. One value expressed by participants was the need for timely resources. We define timeliness as both efficiency in the time taken to complete an action and the appropriate timing of the resource or support. Participants spoke of prioritizing resources, courses, and workshops that saved them time and were critical of resources that were deemed time consuming or inefficient. Additionally, participants wanted their courses and workshops to be well-timed. Courses that addressed current problems were positively reviewed while courses out of sync with participant needs were critiqued.

"It was really beneficial to take the course, it still was really good, but I think it was just not timely." - P12

Participants were not only mindful of their own time, but they were also mindful of the time they asked from others. Peer's time, as well as time from mentors and instructors were considered valuable and participants were often concerned about "burdening" others (P15).

"One challenge is always that you feel like you don't want to take up a lot of the time, especially if they're coworkers, cause you know, they have things to do for work." - P15

While certain resources were deemed timely by some participants and untimely by other participants (e.g. Googling, course notes, asking others for help), the value of timeliness was widely shared by participants.

4.1.2 Awareness. Awareness refers to the ability to know the resource or support exists. The level of reported awareness varied greatly between participants with some feeling "really lucky and well endowed with resources" (P3) while others were not aware that any resources for coding and programming existed at all.

"I don't know, maybe [a coding and programming resource] exists at the U and I just haven't used it." - P13

At minimum, some awareness is required to access any resource for learning. Participants encountered a lack of awareness differently, with some not having looked for resources and others not being able to effectively filter through repositories for resources they believed would be there. Overall, our findings show that the current levels and experiences of awareness of resources is wide-ranging.

4.1.3 Accessibility. We define accessibility as the ability to use a resource or support equitably, if not identically. Similar to levels of awareness, participants reported differing levels of access. Some participants expressed feeling they had access to "everything I can think of that I would want" (P18) while others wanted more access to resources.

A small minority of participants reported a lack of access to the basic hardware and software materials necessary for operating programming systems. For instance, some participants did not have a laptop available when beginning their degree program while others expressed a hesitancy to update old technology because they did not consider themselves "tech-savvy" (P2) enough to manage unfamiliar devices. While a minority of participants faced these issues, their experiences convey important and often overlooked considerations for how universities could effectively support learners.

If participants were able to use the programming systems, they emphasized the benefits of technology-enhanced accessibility practices. Participants listed features such as archived notes, recorded meetings, and shared screens as particularly helpful. They also critiqued in-person settings in which those options were limited.

"In person is so different from zoom. Because [with] zoom most of the time you can screenshot. You can record. You can also share-screen. Those are features that are very helpful for learners." - P16

The accessibility of online resources was not as highly regarded. While participants acknowledged the vast amount of information available on the internet, participants also critiqued the challenge of finding the correct information.

"Well, the online resources, I mean there are a bunch of those. But when it comes to R, again the interpretation of those codings. It's so complicated. And they're like so much, so many." - P5

Participants may have a wide array of information at their fingertips, but still report limited confidence and literacy when choosing the correct resource.

4.2 Transferability

Transferability refers to the ability to apply knowledge gained in one situation appropriately to a new situation. Students need to trust and be trusted to successfully transfer the knowledge they have gained to the correct context. Participants referred to three main components of transferability: specificity, workability, and trustworthiness.

4.2.1 Specificity. We define specificity as the ability of resources and materials to address unique research contexts. Resources were

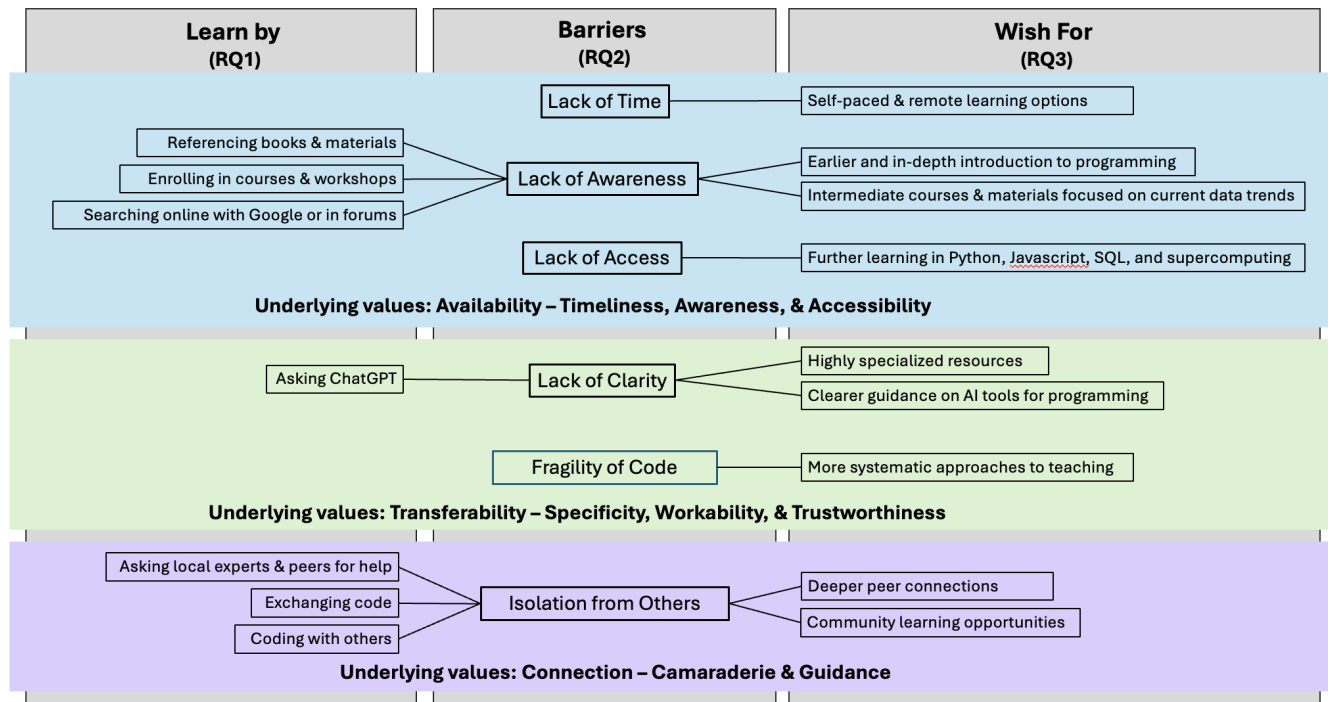


Figure 1: Results of how participants report learning to program, how participants describe barriers, and resources participants wish they could access. Underlying value clusters are highlighted in blue, green, and purple.

often positively regarded for being highly specific or criticized for being too general.

"They're really general courses, and I think, especially being in the social science part of it, a lot of the courses are geared towards more computer science. So like computational modeling stuff, which is not necessarily what I would use it for." - P11

Multiple components of specificity were highlighted including the desire for resources to address unique research needs, especially those for social sciences. These needs could focus on a specific lab or span "overarching comparisons across programs." (P17). At any level, participants called for resources to be more relevant to their research needs.

"The assignment is not related to what's your research project, your own research, dissertation, and thesis topic. So you have to follow the steps and finish and take most of the time working on that course. Eventually you may or may not connect to what you're using. Sometimes you may never use them again." - P16

Participants also spoke to the desire for workshops that focus on more current and attractive visualizations and packages.

"Why don't you do a little bit of ethnography of your students? Why do you offer ggplot? That's the most useless...All work is good, but it could be better employed. Like you could give things that are really attractive." - P6

In terms of curriculum, participants called for courses that align with both introductory and intermediate needs. More senior participants tended to identify a dearth of intermediate course options while novice learners named a desire for more introductory courses earlier in their programs and undergraduate degrees.

"Once you have sort of a basic grasp of underlying concepts, I struggle to find more intermediate level stuff. Which is sort of what I need. Because I am by no means advanced but could use an intermediate help." - P18

"It would be nice if I could have done something like my [intro] R coursera class in my undergrad classes." - P17

Clear understanding of individualized and unique needs from instructors and leaders was sought after by participants as well. Within the classroom, participants asked for clear examples that illustrate processes for their specific contexts.

"You tell me 10 times I might not get it. But if I do it by myself only once, I will get it. Or if I see the image only one time, I'll get it." - P14

Participants also wanted more clarity from leadership in their courses, departments, and institutions. While hearing from others that they must learn on their own, participants challenged this rhetoric by voicing beliefs that faculty and recruiters within their disciplines should be providing further guidance for students.

"...which is why I feel like it adds into the hidden curriculum. [Advisors] kind of already expect you to ask this older student to do it rather than having the University

have resources that can maybe teach you a little better.”
- P10

With more graduate programs and employers seeking programming skills, participants identified the responsibility their recruiters and advisors have to properly prepare prospective and current students. From labs to workshops to course content, participants named over and over again the desire for specific resources and support.

4.2.2 Workability. While specificity may be wanted, it is not sufficient. Participants also named a need for ‘workability’ or the ability for a resource to help their code work. Though not always detailed about what type of help would be needed, participants were clear that the resources most valued were those that led to success.

“Once, I really actually understood, like, how to do loops, I was like, oh, okay, still not the cleanest, best [code] ever right?...But they work for me.” - P13

Frustration was expressed when resources or instructions led to errors, especially when the process suggested by local experts was followed but did not end in the same result.

“Sometimes, even though I write the same data as my professor in R, I receive errors. I cannot proceed from that. My professor’s notes keep going, keep going and I [am] actually stuck at that one point.” - P5

Rather participants requested or wished for support that would verify and validate the workability of their code.

“Some of my friends who are software engineers...talk about this thing called partner coding where 2 people are paired up to do the coding. So you’re essentially coding together...Then somebody else is at least looking over my code and making sure I don’t copy and paste the wrong variable at least.” - P18

In addition to efficiency and replicability, participants also prioritized programming resources that worked.

4.2.3 Trustworthiness. Though participants prioritized code that ran successfully, they were well aware of the limitations of their own and other’s code. While more experienced participants acknowledged they could often get their code to run, they desired to produce and receive code that could be trustworthy - characterized as clean, efficient, and publishable code. They also felt frustration when their code was “very, very gross and hacky” (P18). Participants, especially those in data science positions or near the end of their program, were aware of the expectations of ‘clean’ code in academia and industry and wished for support in maintaining that level of writing.

“People are pushing for more open science like having code available when you publish data and stuff like that. That seems like a good way of starting to hold people accountable for stuff like that. That’s also terrifying, too, that people are gonna be looking through my published code and finding errors. I wish there was a process to do it before I published papers.” - P18

Overall, participants from novices to burgeoning experts in research programming focused primarily on gaining skills in writing and running code. However, as their literacy and awareness

grew, they became more concerned about the professionalism and longevity of people’s code.

“Some publications do include their R code in the publication. And that helps immensely because they will give you the R code, and say, this is how I coded these...It’s not very popular, but it is very helpful.” - P15

Participants appreciated openly published code but found “the worst [challenge] is when a resource or a library gets abandoned” (P6) and were frustrated by “depending on code that is so frail if neglected” (P6). As participants gained more experience coding, their expectations of trustworthiness for their own and others’ code seemed to increase.

4.3 Connection

Connection refers to the ability to build relationships with other peers and experts. Building relationships allows students to share their experiences, problems, and needs as learners. Participants emphasized two main sources of connection: with peers through camaraderie and with experts through guidance.

4.3.1 Camaraderie. Camaraderie describes the ability of peers to relate to participants’ experiences. Needing emotional connection and validation from others, participants often celebrated the connections they made with peers or emotionally spoke of the isolation they felt without support.

“Basically the idea is letting more students see that they are not alone. Most of the time I feel like I am the one only [who] had this problem. But when I ask around, it’s not only me.” - P16

When participants reported feeling isolated, it was attributed to less supportive labs, disconnection from peers, or courses heavily focused on independent work. Participants who felt lucky to be well-connected described building relationships with peers, lab-mates, classmates, and people in their personal life. Those who wished for more connections named peer coding groups and networking spaces as possible opportunities.

“I think it’d be really cool to have peer coding groups on campus. The TA for stats has been so, so helpful and I’m sad that I’m not gonna have that resource after this class is over. But having a peer coding support group, I guess would be really cool.” - P20

Whether the person relating to the participant was also knowledgeable about coding or had no understanding of coding at all, it was helpful to have someone validate the difficult experience of learning to code.

4.3.2 Guidance. When connections were made, guidance from experts, or the ability of someone to pass on their lived experiences and knowledge to others, was also appreciated. Participants often sought out help from experts on specific or difficult problems since they felt it would “solve my problem faster than I would on my own” (P13) and help translate their issues.

“My instructor or my friends, who are more proficient than me, could [use] human language, try to describe what it is that I was trying to do, and they could ask me, “Oh, do you mean this?” Or I could draw if it was a

graph that I was trying to do and they could help me translate into human or into a computer language." - P3

In alignment with participants' value for timeliness and specificity, participants were less likely to go to professors if they felt they might be "burdening" (P15) their time or if the professor might not know the specific context of their question.

"I think the same goes for professors. I know they have open hours, and most professors say that they're open to students coming to them, which I think is true, but I think students still feel like "Oh, but I'm burdening you with things that maybe I should know." And you know, like maybe they're not happy that I don't know about."
- P15

Similarly to the disparity found in levels of awareness of resources, participants reported different levels of connection with experts. Some had guidance from experts in their personal life while others were unable to rely on advisors or instructors.

"But you really have to have good mentors and people that are willing to teach you, because I know it can be frustrating...So I've been lucky in that aspect. But I know that it's not the case for all." - P11

"The kind of attitude that I've encountered with these professors is, "Oh, well, you'll just get it. You're fine." Very dismissive. It's very discouraging, because I don't want to ask for help if I'm telling you the way that would help me best, and you're just dismissing it. Why would I want to ask for help in the future?" - P2

Regardless of the level of connection achieved, participants highlighted the value and desire for expert guidance.

5 Discussion

5.1 Principle Results

Our findings contribute insights into the values driving social science graduate students' needs when learning to program for independent research. Participants highlighted three main values, each with two to three sub-themes. First, participants spoke of the need for availability, or the ability of a resource to be used or accessed. Participants emphasized the need for timeliness, awareness, and accessibility of available resources. Participants also discussed the need for transferability, or the ability to trust and be trusted by others to successfully transfer the knowledge they have gained to the correct contexts. In order to obtain transferability, resources and supports must be specific, workable, and trustworthy. Lastly, learning cannot occur in isolation and participants strongly articulated the need for connection with peers and experts.

Throughout the semi-structured interviews, we also found participants used a variety of resources and strategies including searching on google, online databases, and community forums (e.g. Stack Overflow); referencing books; utilizing ChatGPT; exchanging code with others; programming with others in a group; asking local experts and peers for help; and enrolling in university courses, workshops, and online tutorials. Participants also reported facing multiple barriers to learning including lack of time to engage with resources, lack of awareness of existing resources, a lack of clarity within the classroom and from leadership, inability to access

workable hardware or software, fragile and abandoned code from others, and isolation from peers and experts. In order to enhance their learning, participants wished they could access a more systematic approach to learn programming for research; an earlier and more in depth introduction to programming; intermediate courses and resources focused on current data trends; community learning opportunities; highly specialized resources; self-paced and remote learning options; deeper peer connections; clearer guidance on AI tools for programming; and further learning in Python, Javascript, SQL, and supercomputing.

In alignment with previous research [40, 51], social science graduate students reported learning through independent research projects and seeking connection with others, especially peers and experts. However, these supports were not unanimously available, with some participants reporting almost complete isolation and others reporting feeling "really lucky" (P3) to have many resources. The experiences of our participants were also aligned with previous research that has found that CS major undergraduate students consider coding challenging and express insecurity in their coding skills [33]. As one student provocatively stated when asked how they acquired the coding skills they needed, "a river of tears is a valid answer?" (P6). Yet, unlike CS major students who most commonly express those feelings in introductory courses [36], our participants expressed those concerns in a wide range of skill levels from novice to burgeoning expert. Similar to the expectations of faculty and administrators found by Hennessey et al. [25], participants in our study stated the expectation that they must learn on their own. The acknowledged privilege of some participants and the barriers of others speaks to a wide disparity of access between graduate students in the social sciences when learning to code. Our findings support previous work and provide new insights that begin to elucidate priorities and directions for future designs.

5.2 Methodological Reflections

Before highlighting the implications for future research and pedagogical designs, two important reflections on the method of the study are needed. First, recruitment of participants relied heavily on faculty participation from a department focused on educational psychology. Since the department offers more courses focused on teaching programming than most departments at the institution, our findings may be skewed as compared to the larger social science graduate student population.

Second, three participants were learning to program but had yet to use programming for their own research projects. While their experiences are critical for understanding early novices' programming experiences, we cannot make claims from their experiences about programming to complete independent research. Both of these reflections identify restrictions on the representation of the population and therefore the external validity of the study.

5.3 Implications for Pedagogical Designs

Considering these methodological reflections and restrictions, important implications for pedagogical design arise from our findings. Future pedagogical designs will need to reconcile or prioritize current tensions between values. For instance, the value of trustworthiness can come into conflict with camaraderie. Participants

discussed the need to trust other's code and for others to trust their own code. Within a lab setting, social science graduate students may consider trustworthiness as keeping findings private and withholding procedures until publishing. However, participants also emphasized their desire for camaraderie, or peers relating to their experiences. Hesitations arose as participants voiced worries about seeking validation from peers while also keeping their work private. This tension may be particularly relevant in workshops with other social science graduate students [25, 30, 52] and may limit access to peer code review and feedback strategies often introduced in computer science courses [36]. Any future designs should consider both values and allow social science graduate students to control their level of privacy when connecting with others.

Our findings also illuminate the wide disparities between social science graduate students' levels of awareness of resources and connections to experts. Some participants described themselves as lucky to have many resources and the ability to connect to a local expert such as a friend or advisor with especially strong coding skills. Other participants described little awareness of resources across campus and strong feelings of isolation. The wide range of experiences may be due to the common expectation expressed by participants that they learn on their own, an expectation also expressed in previous studies focused on teaching and learning programming beyond computer science degrees [25, 40]. Future pedagogical designs should consider the limitations of institutional structures for support and identify ways to create parity among social science graduate students.

Lastly, resources and strategies designed to ameliorate social science graduate students' learning experience should focus on the transferability of knowledge gained and connection with others. As programmers, researchers have highly specific end goals that can differ widely from the goals of non-major undergraduate students [29, 53] and counter many of the myths of traditional computing education [47]. Transferability of learning is a long-standing concern in computer science education [5], and may be an even more difficult task for research computing with highly specified disciplinary needs. Participants also called for resources that minimize isolation by supporting connection with peers and experts. Tools such as Thyone [18] (a Jupyter notebook add-on for co-regulation while coding) could be adapted for research computing languages (e.g. R) and further investigated for aiding connection with peers.

5.4 Limitations and Future Work

By identifying common values for support and resources for graduate students in social sciences learning programming for independent research, our study illuminates various limitations and indicates future directions for research. As a qualitative study, we cannot make claims of causality nor can we predict variables that lead to improved learning outcomes. Through reflexive data-driven thematic analysis with inductively clustered codes, our work instead focuses on documenting social science graduate students' experiences and capturing the processes they currently use to learn programming [9]. Future studies could empirically test designed interventions to better determine effective and feasible strategies for improving educational outcomes.

Our study is also limited by the number and breadth of participants. Out of the total number of graduate students who indicated interest in the study, seventeen participants self-reported as eligible and were interviewed. Additionally, psychology students are heavily represented in our study with twelve of the seventeen participants. This extensive representation is most likely due to our targeted recruiting to social science departments with courses that include programming, which is itself overrepresented by psychology disciplines at our university. Future studies could interview participants and deploy questionnaires guided by the questions and values identified in this study to a larger sample size across a wider range of disciplines to better quantify the frequency of graduate students' experiences and increase external validity of the study.

Furthermore, our study focused exclusively on social science graduate student voices and experiences due to the dearth of research focusing on that population. However, there are more community members within the problem space of teaching and learning programming for research. Future studies could bring together multiple types of community members to co-design interventions to address graduate students' concerns and needs when learning. A design community could define, test, and refine interventions from computing education and statistics education for adaptation within the social sciences.

6 Conclusion

Our study contributes to the understanding of how social science graduate students acquire programming skills, what barriers they encounter, and what resources they wish they were able to access. Our study also contributes to the understanding of underlying values social science graduate students hold about the resources and supports available for learning programming for independent research. By elucidating pedagogical design implications and centering social science graduate student experiences, our study grounds future interventions in values expressed by those most impacted by the current limitations of the learning environment.

Acknowledgments

We would like to thank Chase Shanafelt for their invaluable guidance on the clarity and communication of our work.

References

- [1] Murtaza Ali and Sayamindu Dasgupta. 2025. "Even Though I Went Through Everything, I Didn't Feel Like I Learned a Lot": Insights From Experiences of Non-Computer Science Students Learning to Code. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–18. doi:10.1145/3706598.3713624
- [2] Saul Amarel. 1971. Computer science: a conceptual framework for curriculum planning. *Commun. ACM* 14, 6 (June 1971), 391–401. doi:10.1145/362604.362610
- [3] Thea P. Atwood, Andrew T. Creamer, Joshua Dull, Julie Goldman, Kristin Lee, Lora C. Lelidon, and Sarah K. Oelker. 2019. Joining Together to Build More: The New England Software Carpentry Library Consortium. *Journal of eScience Librarianship* 8, 1 (July 2019). doi:10.7191/jeslib.2019.1161 Number: 1 Publisher: Lamar Soutter Library, UMass Chan Medical School.
- [4] John Aycock. 2012. μ Python: non-majors programming from the very first lecture. In *Proceedings of the 17th ACM annual conference on Innovation and technology in computer science education (ITiCSE '12)*. Association for Computing Machinery, New York, NY, USA, 345–350. doi:10.1145/2325296.2325376
- [5] Charles H. Beard. 1993. Transfer of Computer Skills From Introductory Computer Courses. *Journal of Research on Computing in Education* 25, 4 (June 1993), 413–430. doi:10.1080/08886504.1993.10782062 Publisher: Routledge _eprint: <https://doi.org/10.1080/08886504.1993.10782062>.

- [6] Robert E. Beck, Lillian N. Cassel, and Richard H. Austing. 1989. Computer science: a core discipline of liberal arts and sciences. In *Proceedings of the twentieth SIGCSE technical symposium on Computer science education (SIGCSE '89)*. Association for Computing Machinery, New York, NY, USA, 56–60. doi:10.1145/65293.65300
- [7] Caroline Berger, Matthew Lutz, Niklas Elmqvist, Magnus Madsen, and Clemens Nylandsted Klokmose. 2024. Scientists and Code: Programming as a Tool. (May 2024). doi:10.1184/R1/25587726.v1 Publisher: Plateau Workshop.
- [8] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (Jan. 2006), 77–101. doi:10.1191/1478088706qp0630a Publisher: Routledge _eprint: <https://www.tandfonline.com/doi/pdf/10.1191/1478088706qp0630a>.
- [9] Virginia Braun and Victoria Clarke. 2022. Conceptual and Design Thinking for Thematic Analysis. *Qualitative psychology (Washington, D.C.)* 9, 1 (2022), 3–26. doi:10.1037/qup0000196 Publisher: Educational Publishing Foundation.
- [10] Kelly Caine. 2016. Local Standards for Sample Size at CHI. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (CHI '16)*. Association for Computing Machinery, New York, NY, USA, 981–992. doi:10.1145/2858036.2858498
- [11] California Institute of Technology. 2021. Nasa Turns To The Cloud For Help With Next Generation Earth Missions. <https://www.jpl.nasa.gov/news/nasa-turns-to-the-cloud-for-help-with-next-generation-earth-missions/>
- [12] Yingjun Cao, Leo Porter, and Daniel Zingaro. 2016. Examining the Value of Analogies in Introductory Computing. In *Proceedings of the 2016 ACM Conference on International Computing Education Research (ICER '16)*. Association for Computing Machinery, New York, NY, USA, 231–239. doi:10.1145/2960310.2960313
- [13] Ralph W. Crosby, Stephanie Valentine, and Tiffani L. Williams. 2014. Leveraging programming difficulty to improve understanding and perceptions of non-majors. *J. Comput. Sci. Coll.* 29, 4 (April 2014), 27–35.
- [14] Jessica Q. Dawson, Meghan Allen, Alice Campbell, and Anasazi Valair. 2018. Designing an Introductory Programming Course to Improve Non-Majors' Experiences. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education (SIGCSE '18)*. Association for Computing Machinery, New York, NY, USA, 26–31. doi:10.1145/3159450.3159548
- [15] Max Fowler, Binglin Chen, and Craig Zilles. 2021. How should we 'Explain in plain English'? Voices from the Community. In *Proceedings of the 17th ACM Conference on International Computing Education Research (ICER 2021)*. Association for Computing Machinery, New York, NY, USA, 69–80. doi:10.1145/3446871.3469738
- [16] Patricia Fusch and Lawrence Ness. 2015. Are We There Yet? Data Saturation in Qualitative Research. *The Qualitative Report* 20, 9 (Sept. 2015), 1408–1416. doi:10.46743/2160-3715/2015.2281
- [17] Norman E. Gibbs and Allen B. Tucker. 1986. A model curriculum for a liberal arts degree in computer science. *Commun. ACM* 29, 3 (March 1986), 202–210. doi:10.1145/5666.5667
- [18] Lahari Goswami, Alexandre Senges, Thibault Estier, and Mauro Cherubini. 2023. Supporting Co-Regulation and Motivation in Learning Programming in Online Classrooms. *Proc. ACM Hum.-Comput. Interact.* 7, CSCW2 (Oct. 2023), 298:1–298:29. doi:10.1145/3610089
- [19] Luke Gusukuma, Austin Cory Bart, Dennis Kafura, and Jeremy Ernst. 2018. Misconception-Driven Feedback: Results from an Experimental Study. In *Proceedings of the 2018 ACM Conference on International Computing Education Research (ICER '18)*. Association for Computing Machinery, New York, NY, USA, 160–168. doi:10.1145/3230977.3231002
- [20] Mark Guzdial, Emma Dadoo, Bahare Naimpour, Tamara Nelson-Fromm, and Aadash Padiyath. 2023. Putting a Teaspoon of Programming into Other Subjects. *Commun. ACM* 66, 5 (April 2023), 24–26. doi:10.1145/3587926
- [21] Mark Guzdial and August Evrard. 2024. Identifying the Computing Education Needs of Liberal Arts and Sciences Students (Discussion Paper). In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3631802.3631805
- [22] Mark Guzdial and Andrea Forte. 2005. Design process for a non-majors computing course. In *Proceedings of the 36th SIGCSE technical symposium on Computer science education (SIGCSE '05)*. Association for Computing Machinery, New York, NY, USA, 361–365. doi:10.1145/1047344.1047468
- [23] Rogers Hall and A. Susan Jurow. 2015. Changing Concepts in Activity: Descriptive and Design Studies of Consequential Learning in Conceptual Practices. *Educational Psychologist* 50, 3 (July 2015), 173–189. doi:10.1080/00461520.2015.1075403 Publisher: Routledge _eprint: <https://doi.org/10.1080/00461520.2015.1075403>.
- [24] Mohammed Hassan, Grace Zeng, and Craig Zilles. 2024. Evaluating How Novices Utilize Debuggers and Code Execution to Understand Code. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1 (ICER '24, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 65–83. doi:10.1145/3632620.3671126
- [25] Cody Hennesy, Caitlin Bakker, Nicholas J. H. Dunn, David Naughton, Franklin Sayre, Stacie Traill, Cody Hennesy, Caitlin Bakker, Nicholas J. H. Dunn, David Naughton, Franklin Sayre, and Stacie Traill. 2022. Building, Sustaining, and Growing Multidisciplinary, Multi-Departmental Partnerships to Teach Open Science Tools. In *Handbook of Research on Academic Libraries as Partners in Data Science Ecosystems*. IGI Global, 169–188. <https://www.igi-global.com/gateway/chapter/www.igi-global.com/gateway/chapter/302753> ISBN: 9781799897026 Publisher: IGI Global.
- [26] Felienne Hermans. 2020. Hedy: A Gradual Language for Programming Education. In *Proceedings of the 2020 ACM Conference on International Computing Education Research*. ACM, Virtual Event New Zealand, 259–270. doi:10.1145/3372782.3406262
- [27] Bradford W. Hesse, Richard P. Moser, and William T. Riley. 2015. From Big Data to Knowledge in the Social Sciences. *The ANNALS of the American Academy of Political and Social Science* 659, 1 (May 2015), 16–32. doi:10.1177/0002716215570007 Publisher: SAGE Publications Inc.
- [28] Emma Hogan, Ruoxuan Li, and Adalbert Gerald Soosai Raj. 2023. CS0 vs. CS1: Understanding Fears and Confidence amongst Non-majors in Introductory CS Courses. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2023)*. Association for Computing Machinery, New York, NY, USA, 25–31. doi:10.1145/3545945.3569865
- [29] Jinyoung Hur and Kathryn Cunningham. 2024. Profiling Conversational Programmers at University: Insights into their Motivations and Goals from a Broad Sample of Non-Majors. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1 (ICER '24, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 293–311. doi:10.1145/3632620.3671123
- [30] Amy L. Johnson and Rebecca D. Gleit. 2022. Teaching for a Data-Driven Future: Intentionally Building Foundational Computing Skills. *Teaching Sociology* 50, 1 (Jan. 2022), 49–61. doi:10.1177/0092055X211033632 Publisher: SAGE Publications Inc.
- [31] Panagiotis Kampylis, Valentina Dagienė, Stefania Bocconi, Augusto Chiocciariello, Katja Engelhardt, Gabriel Stupurienė, Vaida Masilionytė-Dagienė, Eglė Jasutė, Chiara Malagoli, Milena Horvath, and Jeffrey Earp. 2023. Integrating Computational Thinking into Primary and Lower Secondary Education: A Systematic Review. *Educational Technology & Society. Educational Technology & Society* 26, 2 (April 2023), 99–117. doi:10.30191/ETS.202304_26(2).0008 Publisher: International Forum of Educational Technology & Society (IFETS).
- [32] Jae Yeon Kim and Yee Man Margaret Ng. 2022. Teaching Computational Social Science for All. *PS, political science & politics* 55, 3 (2022), 605–609. doi:10.1017/S1049096521001815 Place: New York, USA Publisher: Cambridge University Press.
- [33] Essi Lahtinen, Kirsti Ala-Mutka, and Hannu-Matti Järvinen. 2005. A study of the difficulties of novice programmers. In *Proceedings of the 10th annual SIGCSE conference on Innovation and technology in computer science education (ITICSE '05)*. Association for Computing Machinery, New York, NY, USA, 14–18. doi:10.1145/1067445.1067453
- [34] Sam Lau, Deborah Nolan, Joseph Gonzalez, and Philip J. Guo. 2022. How Computer Science and Statistics Instructors Approach Data Science Pedagogy Differently: Three Case Studies. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1 (SIGCSE 2022, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 29–35. doi:10.1145/3478431.3499384
- [35] Yimiao Li, Melissa Chen, Ayse Hunt, Haoqi Zhang, and Eleanor O'Rourke. 2024. Exploring the Interplay of Metacognition, Affect, and Behaviors in an Introductory Computer Science Course for Non-Majors. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1 (ICER '24, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 27–41. doi:10.1145/3632620.3671119
- [36] Andrew Luxton-Reilly, Simon, Ibrahim Albluwi, Brett A. Becker, Michail Giannakos, Amruth N. Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory programming: a systematic literature review. In *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (ITICSE 2018 Companion)*. Association for Computing Machinery, New York, NY, USA, 55–106. doi:10.1145/3293881.3295779
- [37] Samiha Marwan, Ge Gao, Susan Fisk, Thomas W. Price, and Tiffany Barnes. 2020. Adaptive Immediate Feedback Can Improve Novice Programming Engagement and Intention to Persist in Computer Science. In *Proceedings of the 2020 ACM Conference on International Computing Education Research (ICER '20)*. Association for Computing Machinery, New York, NY, USA, 194–203. doi:10.1145/3372782.3406264
- [38] Jessica K. Miller, Batya Friedman, Gavin Jancke, and Brian Gill. 2007. Value tensions in design: the value sensitive design, development, and appropriation of a corporation's groupware system. *ACM, New York, NY, USA*, 281–290. doi:10.1145/1316624.1316668 Book Title: Conference on Supporting Group Work: Proceedings of the 2007 international ACM conference on Conference on supporting group work; 04-07 Nov. 2007.
- [39] Deborah Nolan and Duncan Temple Lang. 2010. Computing in the Statistics Curricula. *The American Statistician* 64, 2 (2010), 97–107. <https://www.jstor.org/stable/20799880> Publisher: [American Statistical Association, Taylor & Francis, Ltd.].
- [40] Elle O'Brien and Jordan Mick. 2024. In the Academy, Data Science Is Lonely: Barriers to Adopting Data Science Methods for Scientific Research. *Harvard Data Science Review* 6, 2 (April 2024). doi:10.1162/99608f92.7ca04767 Publisher: The

- MIT Press.
- [41] Laura L. Paglis, Stephen G. Green, and Talya N. Bauer. 2006. Does adviser mentoring add value? A longitudinal study of mentoring and doctoral student outcomes. *Research in Higher Education* 47, 4 (June 2006), 451–476. doi:10.1007/s11162-005-9003-2
 - [42] Kang-il Park, Pierre Weill-Tessier, Neil C. C. Brown, Bonita Sharif, Nikolaj Jensen, and Michael Kölling. 2023. An eye tracking study assessing the impact of background styling in code editors on novice programmers' code understanding. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1 (ICER '23, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 444–463. doi:10.1145/3568813.3600133
 - [43] Thomas W. Price and Tiffany Barnes. 2015. Comparing Textual and Block Interfaces in a Novice Programming Environment. In *Proceedings of the eleventh annual International Conference on International Computing Education Research (ICER '15)*. Association for Computing Machinery, New York, NY, USA, 91–99. doi:10.1145/2787622.2787712
 - [44] Timothy Rafalski, P. Merlin Uesbeck, Cristina Panks-Meloney, Patrick Daleiden, William Allee, Amelia Mcnamara, and Andreas Stefik. 2019. A Randomized Controlled Trial on the Wild Wild West of Scientific Computing with Student Learners. In *Proceedings of the 2019 ACM Conference on International Computing Education Research (ICER '19)*. Association for Computing Machinery, New York, NY, USA, 239–247. doi:10.1145/3291279.3339421
 - [45] Savitha Ravi, Sai Krishna Ranjan Gauravarapu, and Michael Coblenz. 2024. A Preliminary Study of the Needs of Climate Data Users. (May 2024). doi:10.1184/R1/25586877.v1 Publisher: Plateau Workshop.
 - [46] Ryan Rybarczyk. 2020. Non-Major Peer Mentoring for CS1. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE '20)*. Association for Computing Machinery, New York, NY, USA, 1068–1074. doi:10.1145/3328778.3366901
 - [47] Mary Shaw. 2022. Myths and mythconceptions: what does it mean to be a programming language, anyhow? *Proc. ACM Program. Lang.* 4, HOPL (April 2022), 234:1–234:44. doi:10.1145/3480947
 - [48] Beth Simon, Brian Hanks, Renée McCauley, Briana Morrison, Laurie Murphy, and Carol Zander. 2009. For me, programming is In *Proceedings of the fifth international workshop on Computing education research workshop (ICER '09)*. Association for Computing Machinery, New York, NY, USA, 105–116. doi:10.1145/1584322.1584335
 - [49] K. Bret Staudt Willet and Joshua M. Rosenberg. 2023. The Design and Effects of Educational Data Science Workshops for Early Career Researchers. *Journal of formative design in learning* 7, 2 (2023), 83–97. doi:10.1007/s41686-023-00083-7 Place: Cham Publisher: Springer International Publishing.
 - [50] James D. Teresco, Andrea Tartaro, Amanda Holland-Minkley, Grant Braught, Jakob Barnard, and Douglas Baldwin. 2022. CS Curricular Innovations with a Liberal Arts Philosophy. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1 (SIGCSE 2022, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 537–543. doi:10.1145/3478431.3499329
 - [51] Allison Theobald and Stacey Hancock. 2019. How Environmental Science Graduate Students Acquire Statistical Computing Skills. *Statistics Education Research Journal* 18, 2 (Nov. 2019), 68–85. doi:10.52041/serj.v18i2.141
 - [52] Allison Theobald, Stacey Hancock, and Sara Mannheimer. 2021. Designing Data Science Workshops for Data-Intensive Environmental Science Research. *Journal of Statistics and Data Science Education* 29, sup1 (Jan. 2021), S83–S94. doi:10.1080/10691898.2020.1854636 Publisher: Taylor & Francis _eprint: <https://doi.org/10.1080/10691898.2020.1854636>.
 - [53] Mike Tissenbaum, David Weintrop, Nathan Holbert, and Tamara Clegg. 2021. The case for alternative endpoints in computing education. *British Journal of Educational Technology* 52, 3 (May 2021), 1164–1177. doi:10.1111/bjet.13072 Publisher: John Wiley & Sons, Ltd.
 - [54] Wengran Wang, Rui Zhi, Alexandra Milliken, Nicholas Lytle, and Thomas W. Price. 2020. Crescendo: Engaging Students to Self-Paced Programming Practices. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE '20)*. Association for Computing Machinery, New York, NY, USA, 859–865. doi:10.1145/3328778.3366919